

ENG442

Advanced Digital Electronics

Project - RISC-V Softcore Implementation in Verilog

Tyler Robards - 651790, Baley Eccles 652137

1 Introduction

2 Technical Detail

2.1 Design Overview

The processor was designed as a single-cycle RV32I datapath. In a single-cycle design, each instruction completes in one clock cycle. This means that instruction, fetch, decode, register access, ALU execution, memory access, write-back and program counter update all occur during one clock period. This causes the maximum clock speed to be limited by the longest combinational path through the processor.

The top-level module is `riscv_cpu.v`. This module connects the program counter, instruction memory, control unit, register file, immediate generator, ALU control, ALU, data memory, write-back multiplexer and next-PC logic. The supported instruction set includes R-type arithmetic instruction, I-type immediate arithmetic and load instructions, S-type store instructions, B-type branch instructions and J-type jump instructions.

2.2 Datapath Block Diagram

2.3 Module Hierarchy

The CPU uses a top-down module structure. The `riscv_cpu` module is the top-level design and instantiates all of the main datapath modules. The test bench instantiates `riscv_cpu` as the unit under test.

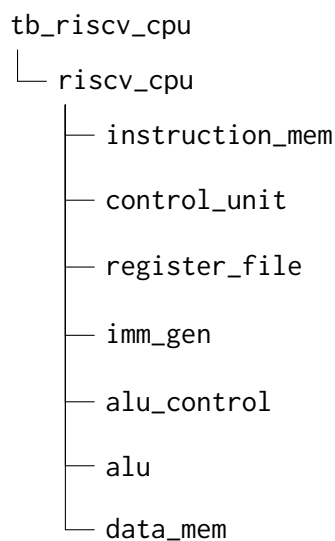


Figure 1: RISC-V CPU module hierarchy.

2.4 Module Descriptions

`riscv_cpu.v`

The `riscv_cpu` module is the top-level module. It contains the 32-bit program counter and connects all of the supporting datapath modules. The current program counter value is used to read an instruction from instruction memory. The instruction is then split into the opcode, destination register, source registers, `funct3` and `funct7` fields. The fields are used by the control unit and ALU control logic to determine the operation required for the current instruction.

The CPU also contains the write-back and next-PC selection logic. For normal ALU instruction, the write-back data is the ALU result. For `lw` the write-back data comes from data memory. For `jal`, the destination register receives $PC + 4$, the branch target, or the jump target depending on the branch and jump control signals.

`control_unit.v`

The control unit decodes the 7-bit opcode and generates the main datapath and control signals. These include `reg_write`, `mem_write`, `mem_read`, `alu_src`, `mem_to_reg`, `branch`, `jump` and `alu_op`. For example, R-type instructions enable register write-back and use the two register operands as ALU inputs. Load instructions enable memory reading, select the immediate as the second ALU input and select memory data for write-back. Store instructions enable memory writing, while branch and jump instructions control the next-PC path.

`alu_control.v`

The ALU control module converts the higher-level `alu_op` signal and instruction function fields into the specific ALU operation. For this project, only addition and subtraction were required. Addition is used for `add`, `addi`, `lw` and `sw`, while subtraction is used for `sub` and `beq`. For R-type instructions, the module checks the combined `funct7` and `funct3` fields to distinguish between `add` and `sub`.

`alu.v` The ALU performs 32-bit addition or subtraction depending on the ALU control signal. It also produces a zero flag when the ALU result is equal to zero. This flag is used by the branch logic for `beq`. When two operands are equal, subtracting them gives zero, causing the branch condition to be true.

`register_file.v`

The register file contains 32 general-purpose 32-bit registers. It has two asynchronous read ports for `rs1` and `rs2` and one synchronous write port for `rd`. The register file is reset to zero at the start of simulation. Register `x0` is forced to remain zero, matching the RISC-V specification.

`imm_gen.v`

The immediate generator extracts and sign-extends immediate values from the instruction word. It supports the immediate formats required for the project: I-type for `addi` and `lw`, S-type for `sw`, B-type for `beq` and J-type for `jal`. Branch and jump immediates include a final zero bit because their target offsets are aligned to two-byte boundaries.

`instruction_mem.v`

The instruction memory stores the assembled program from `program.hex`. It contains 256 32-bit words and uses word-aligned addressing by dividing the byte address by four. At simulation start `$readmeh` loads the hexadecimal machine code into the memory array.

`data_mem.v`

The data memory contains 256 32-bit words. It support synchronous writes and asynchronous reads. The address is word-aligned using `addr[31:2]`, which allows load and store instructions to access word locations using byte addresses from the ALU.

`tb_riscv_cpu.v`

The test bench generates a 100 MHz simulation clock with a 10 ns period. It applies reset, runs the CPU for enough time to execute the test program and then displays the contents of the registers and data memory. This allows the final register and memory values to be checked against the expected results from the assembly program.

2.5 Supported Instruction Behaviour

Table 1: Supported RISC-V instructions in the implemented processor

Instruction	Type	Main behaviour in this design
<code>add</code>	R-type	Adds two register operands and writes the result to <code>rd</code> .
<code>sub</code>	R-type	Subtracts <code>rs2</code> from <code>rs1</code> and writes the result to <code>rd</code> .
<code>addi</code>	I-type	Adds a sign-extended immediate to <code>rs1</code> and writes the result to <code>rd</code> .
<code>lw</code>	I-type load	Uses the ALU to calculate an address, reads data memory, and writes the loaded value to <code>rd</code> .
<code>sw</code>	S-type store	Uses the ALU to calculate an address and stores <code>rs2</code> into data memory.
<code>beq</code>	B-type branch	Subtracts the two register operands and branches when the result is zero.
<code>jal</code>	J-type jump	Jumps to the target address and writes <code>PC + 4</code> to <code>rd</code> .

2.6 FPGA Resource and Performance Considerations

The design is small compared with the resources available on the Basys 3 development board. The main hardware resources used by this processor are flip-flops for the program counter and register file, LUTs for the control logic, multiplexers, immediate generation, ALU logic and data memory resources for instruction and data memory. The instruction memory and data memory are each implemented as 256 words of 32 bits, giving 8192 bits each. Together, they require 16384 bits, not including the register file. This is well within the block RAM capacity of the Basys 3 Artix-7 FPGA.

The processor has a CPI of 1 because it is a single-cycle design. With the 100 MHz test bench clock, the simulated clock period is 10 ns, so the ideal instruction throughput is one instruction every 10 ns. However, for the real FPGA implementation, the maximum clock frequency would depend on the longest combinational path. The likely critical path is from the program counter through instruction memory. decode/control, register file and immediate generation, the ALU, possible data memory access, write-back selection and into the destination register. Because all instruction work must complete within one cycle, a single-cycle CPU is generally slower than a pipelined design for more complex instruction sets.

For this small instruction subset, the Basys 3 FPGA is suitable. The design does not use multiplication, division, floating-point operations, caches, or complex peripherals, so it should require only a small fraction of the available LUTs, flip-flops and memory resources. If implemented on the board, the main practical limitations would be timing closure at the selected clock

frequency, memory inference style and any additional input/output hardware added around the CPU.

3 Testing

3.1 Test Programme

A RISC-V assembly test program was written to check all required instructions. The programme first loads two immediate values into registers, then performs addition and subtraction. It stores the result of the addition into data memory, loads it back into another register, checks equality using `beq` and finally checks jump behaviour using `jal`. The expected results can be seen in Table 2.

Table 2: Instruction test cases and expected results

Instruction tested	Test method	Expected result
<code>addi</code>	Load constants 5 and 10 into registers.	$x1 = 5, x2 = 10$.
<code>add</code>	Add $x1$ and $x2$.	$x3 = 15$.
<code>sub</code>	Subtract $x1$ from $x2$.	$x4 = 5$.
<code>sw</code>	Store $x3$ into data memory address 0.	<code>memory[0] = 15</code> .
<code>lw</code>	Load data memory address 0 into $x5$.	$x5 = 15$.
<code>beq</code>	Compare $x3$ and $x5$.	Branch is taken because both are 15.
<code>jal</code>	Jump to <code>jump_target</code> and store return address.	$x7 = PC + 4$; skipped instruction does not execute.

3.2 Test Bench Method

3.3 Simulation Results

4 Conclusion

5 Code Listings